



本小节内容

各种变量赋值汇编实战解析

1 各种变量赋值汇编实战解析

我们针对整型，整型数组，整型指针变量的赋值（浮点与字符等价的），对应的汇编进行解析，首先我们编写下面 C 代码：

```
#include <stdio.h>
```

```
int main() {
```

```
    int arr[3] = {1, 2, 3};
```

```
    int *p;
```

```
    int i = 5;
```

```
    int j = 10;
```

```
    i = arr[2];
```

```
    p = arr;
```

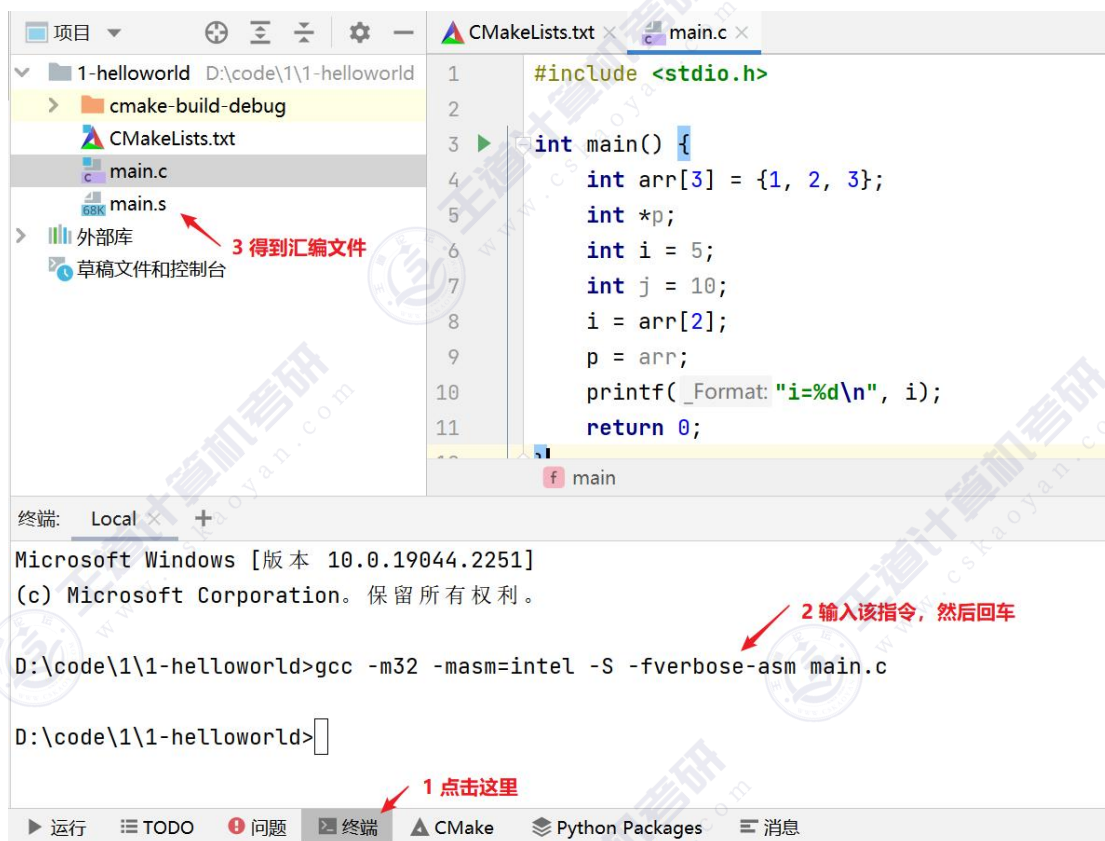
```
    printf("i=%d\n", i);
```

```
    return 0;
```

```
}
```

然后根据上一小节生成汇编的方法，在终端窗口输入

gcc -m32 -masm=intel -S -fverbose-asm main.c（苹果的 M1 电脑如果不可用，可以直接看课件我的汇编即可，也可以去掉-masm=intel，这样汇编和考研大题格式有差异），如下图所示，得到汇编文件 main.s



接下来我们来分析转换后的汇编代码，首先#号代表注释，我们从 main 标签位置开看即可。我们的 C 代码在让 CPU 去运行时，其实所有的变量名都已经消失了，实际是数据从一个空间，拿到另一个空间的过程。

栈内变量先定义在低地址还是高地址，取决于操作系统与 CPU 的组合，你的可能和我的不一样，因此不用去研究，没有意义（也不属于考研大纲范围）。

我们访问所有变量的空间都是通过栈指针（esp 时刻都存着栈指针，也可以称为栈顶指针）的偏移，来获取对应变量内存空间的数据的。

.text

.def __main; .scl 2; .type 32; .endef

.section .rdata,"dr"

LC0:

.ascii "i=%d\n"

.text

.globl _main

.def __main; .scl 2; .type 32; .endef



`_main:`

`push ebp #`

`mov ebp, esp #,`

`and esp, -16 #,`

`sub esp, 48 #,`

`# main.c:3: int main() {`

`call ____main # 从 main 到这里这一段我们函数调用时解析，暂时不管`

`# main.c:4: int arr[3] = {1, 2, 3};`

`mov DWORD PTR [esp+24], 1 # arr, #把常量 1 放入栈指针(esp 寄存器存的栈指针)偏移 24 个字节的`

`mov DWORD PTR [esp+28], 2 # arr,`

`mov DWORD PTR [esp+32], 3 # arr,`

`# main.c:6: int i = 5;`

`mov DWORD PTR [esp+44], 5 # i, #把常量 5 放入栈指针(esp 寄存器存的栈指针)偏移 44 个字节的`
位置，这个位置是变量 i 的空间

`# main.c:7: int j = 10;`

`mov DWORD PTR [esp+40], 10 # j, #把常量 10 放入栈指针(esp 寄存器存的栈指针)偏移 40`

个字节的

`# main.c:8: i = arr[2];`

`mov eax, DWORD PTR [esp+32] # tmp89, arr #把栈指针(esp 寄存器存的栈指针)偏移 32 个字节的`
位置拿到的数据，放到寄存器 eax 中

`mov DWORD PTR [esp+44], eax # i, tmp89 #把 eax 寄存器的内容放入栈指针(esp 寄存器存的栈指针)偏移 44 个字节的`
位置

`# main.c:9: p = arr;`



```
lea eax, [esp+24] # tmp90, #lea 和 mov 不一样, 是拿栈指针偏移 24 个字节的地址,
```

把地址放到 `eax` 寄存器

```
mov DWORD PTR [esp+36], eax # p, tmp90 #把 eax 寄存器的内容放入栈指针(esp 寄存器  
存的栈指针)偏移 36 个字节的位置, 在这里你可以看到
```

```
# main.c:10: printf("i=%d\n", i);
```

```
mov eax, DWORD PTR [esp+44] # tmp91, i #把栈指针, 偏移 44 个字节的位置拿到的数据,  
放到寄存器 eax 中
```

```
mov DWORD PTR [esp+4], eax #, tmp91 #把 eax 数据放到栈指针偏移 4 个字节位置的内存中
```

```
mov DWORD PTR [esp], OFFSET FLAT:LC0 #, #把 LC0(也就是上面那个字符串) 的地址,  
放到寄存器栈指针指向的内存位置
```

```
call _printf # #调用 printf 函数
```

```
# main.c:11: return 0; #下面的内容暂时不管
```

```
mov eax, 0 # _10,
```

```
# main.c:12: }
```

```
leave
```

```
ret
```

```
.ident "GCC: (x86_64-posix-sjlj-rev0, Built by MinGW-W64 project) 8.1.0"
```

```
.def _printf; .scl 2; .type 32; .endef
```

大家转的汇编的偏移值可能和我这里有一些差异, 这个没关系, 这一节大家理解变量赋值的汇编指令及原理即可, 主要掌握的指令是 `mov`, 还有 `lea`, 还有 `PTR`。下面是 `ptr` 介绍

`ptr` -- pointer (既指针) 得缩写。

汇编里面 `ptr` 是规定的字 (既保留字), 是用来临时指定类型的。

(可以理解为, ptr 是临时的类型转换, 相当于 C 语言中的强制类型转换)

如 `mov ax,bx`; 是把 BX 寄存器“里”的值赋予 AX, 由于二者都是寄存器, 长度已定 (word 型), 所以没有必要加 “WORD”

`mov ax,word ptr [bx]`; 是把内存地址等于“BX 寄存器的值”的地方所存放的数据, 赋予 ax。由于只是给出一个内存地址, 不知道希望赋予 ax 的, 是 byte 还是 word, 所以可以用 word 明确指出; 如果不用, 既 (`mov ax,[bx]`;) 则在 8086 中是默认传递一个字, 既两个字节给 ax。

intel 中的

dword ptr 长字 (4 字节)

word ptr 是双字

byte ptr 是一字节